PYTHON ACTIVITY 1
CASE STUDIES 41-50

Caes-41 — Nested List Classroom Occupancy Analysis

- **Problem Statement:**
- Represent classrooms and seats using nested lists. Allocate students while keeping
- reserved seats unchanged, detect duplicate student
- assignments, and calculate vacant capacity.

Objectives

- Store classrooms and seats as nested lists.
- Allocate students only to free, non-reserved seats.
- Detect duplicate student IDs.
- Calculate total, occupied, and vacant seats.

Assumptions

- Each inner list represents one classroom.
- R means a reserved seat; None means a free seat.
- Each student should receive only one seat.
- Students are assigned in input order.

Algorithms

1.Create nested classroom seat lists and student list.

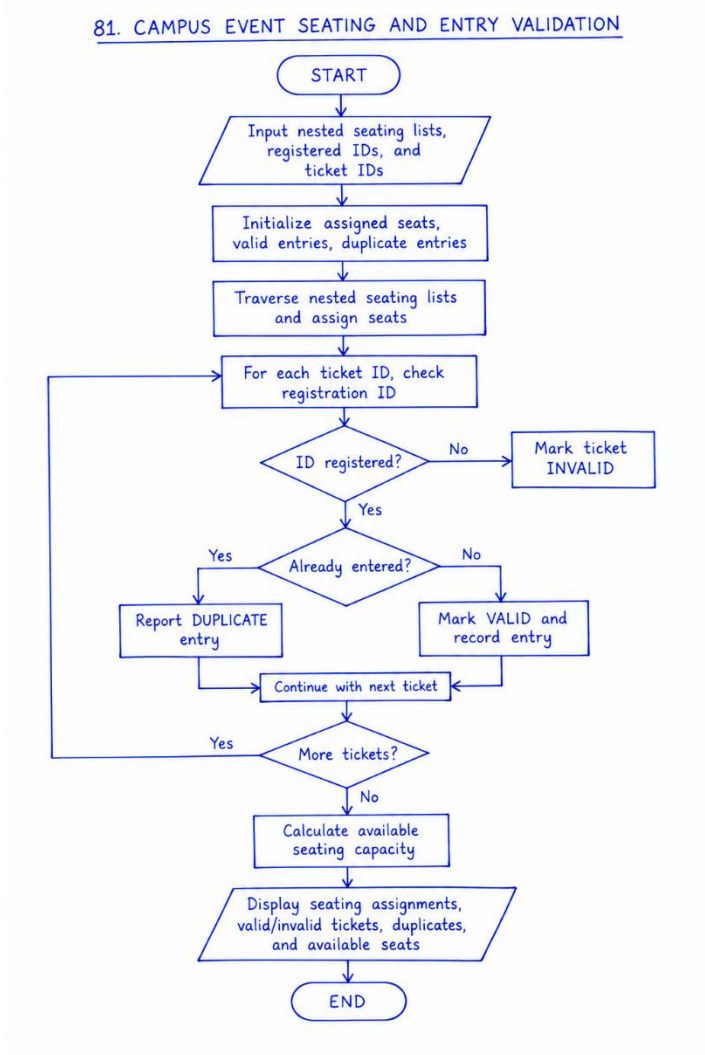
2. Check the student list for duplicates.

3. Traverse every classroom and seat.

4. Skip reserved/occupied seats; assign the next unique student to a free seat.

5. Count total, occupied and vacant seats; display duplicates.

Flow Chart


81. CAMPUS EVENT SEATING AND ENTRY VALIDATION
START
Input nested seating lists, registered IDs, and ticket IDs
Initialize assigned seats, valid entries, duplicate entries
Traverse nested seating lists and assign seats
For each ticket ID, check registration ID
ID registered?
No
Mark ticket INVALID
Yes
Already entered?
Yes
No
Report DUPLICATE entry
Mark VALID and record entry
Continue with next ticket
More tickets?
Yes
No
Calculate available seating capacity
Display seating assignments, valid/invalid tickets, duplicates, and available seats
END

Python Source Code

```
classrooms = [
 [None, "R", None, None],
 [None, None, "R", None]
]
students = ["Asha", "Bala", "Asha", "Cathy"]
duplicates = [s for s in set(students) if students.count(s) > 1]
unique = []
for s in students:
 if s not in unique:
 unique.append(s)
i = 0
for room in classrooms:
 for j in range(len(room)):
 if i < len(unique) and room[j] is None:
 room[j] = unique[i]
 i += 1
total = sum(len(r) for r in classrooms)
occupied = sum(x not in (None, "R") for r in classrooms for x in r)
vacant = sum(x is None for r in classrooms for x in r)
print("Classrooms:", classrooms)
print("Duplicate students:", duplicates)
print("Vacant seats:", vacant)
```

Test Cases

- Test Cases

1. Asha appears twice → Duplicate detected.
2. R seat → Preserved.
3. 8 seats, 3 assigned → Vacant seats shown.

Code And Output

CODE

```
def assign_seats(seating_rows, attendees):
    assignments = {}
    index = 0

    for row_number, row in enumerate(
        seating_rows, start=1
    ):
        for seat in row:
            if index < len(attendees):
                assignments[attendees[index]] = (
                    f"Row {row_number}, Seat {seat}"
                )
                index += 1

    return assignments


def validate_entries(registered_ids, ticket_ids):
    registered = set(registered_ids)
    seen = set()
    valid = []
    invalid = []
    duplicates = []

    for ticket in ticket_ids:
        if ticket not in registered:
            invalid.append(ticket)
        elif ticket in seen:
            duplicates.append(ticket)
        else:
            valid.append(ticket)
            seen.add(ticket)

    return valid, invalid, duplicates


def main():
    seating_rows = [
        ["A1", "A2", "A3", "A4"],
        ["B1", "B2", "B3", "B4"],
        ["C1", "C2", "C3", "C4"]
    ]

    registered_ids = [
        "REG101", "REG102", "REG103",
        "REG104", "REG105"
    ]

    ticket_ids = [
```

OUTPUT

```
CAMPUS EVENT SEATING AND ENTRY VALIDATION
\nSEATING ASSIGNMENTS
REG101: Row 1, Seat A1
REG102: Row 1, Seat A2
REG103: Row 1, Seat A3
REG104: Row 1, Seat A4
REG105: Row 2, Seat B1
\nENTRY VALIDATION
Valid entries: ['REG101', 'REG102', 'REG104']
Invalid tickets: ['REG999']
Duplicate entries: ['REG101']
\nSEATING CAPACITY
Total seats: 12
Assigned seats: 5
Available seats: 7
>>
```

Conclusion

- The program uses nested lists, loops and conditions to allocate seats safely, preserve reserved seats, detect duplicates and find vacant capacity.

CASE-42 — Student Marks Matrix Analysis

Problem statement

- **Analyse marks stored as a matrix where rows are students and columns are subjects.**
- **Find row averages, subject averages, the top**
- **performer and subjects with low average marks.**

Objectives

- Store marks in a matrix.
- Calculate each student's average.
- Calculate each subject's average.
- Find the top student and low-average subjects.

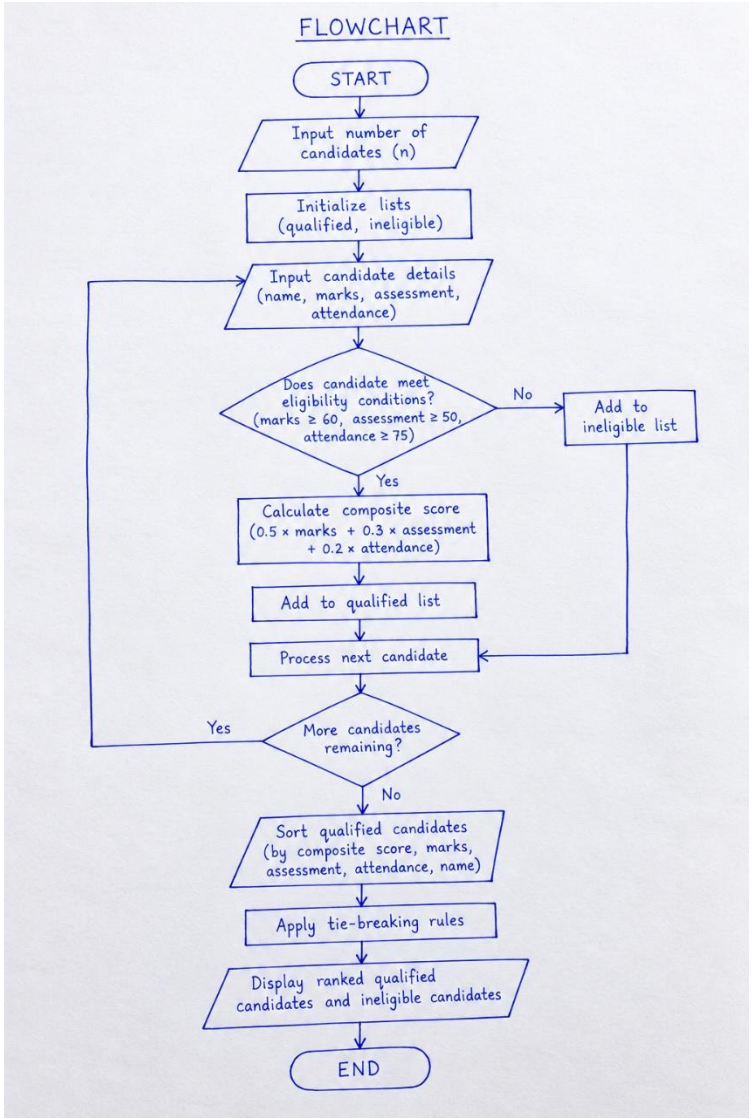
Assumptions

- Each mark is between 0 and 100.
- Each row represents one student.
- Each column represents one subject.
- Low subject average means below 60.

Algorithm

1. Create the marks matrix and subject names.
2. Calculate each row average.
3. Calculate each column average.
4. Find the student with maximum average.
5. List subjects whose average is below 60.

Flowchart:


FLOWCHART
START
Input number of candidates (n)
Initialize lists (qualified, ineligible)
Input candidate details (name, marks, assessment, attendance)
Does candidate meet eligibility conditions? (marks ≥ 60, assessment ≥ 50, attendance ≥ 75)
No
Add to ineligible list
Yes
Calculate composite score (0.5 × marks + 0.3 × assessment + 0.2 × attendance)
Add to qualified list
Process next candidate
More candidates remaining?
Yes
No
Sort qualified candidates (by composite score, marks, assessment, attendance, name)
Apply tie-breaking rules
Display ranked qualified candidates and ineligible candidates
END

```
marks = [
 [80, 70, 90],
 [60, 65, 55],
 [95, 88, 92]
]
students = ["Asha", "Bala", "Cathy"]
subjects = ["Math", "Science", "English"]
row_avg = [sum(r)/len(r) for r in marks]
col_avg = [
 sum(marks[i][j] for i in range(len(marks))) / len(marks)
 for j in range(len(subjects))
]
top = students[row_avg.index(max(row_avg))]
low = [subjects[i] for i, a in enumerate(col_avg) if a < 60]
print("Student averages:", dict(zip(students, row_avg)))
print("Subject averages:", dict(zip(subjects, col_avg)))
print("Top performer:", top)
print("Low-average subjects:", low)
```

Test Cases

1. Asha marks 80,70,90 → Average 80.
2. Cathy has highest average → Top performer Cathy.
3. Any subject average < 60 → Listed as low.

CODE and Output

CODE

```
def is_eligible(marks, assessment, attendance):
    return (
        marks >= 60
        and assessment >= 50
        and attendance >= 75
    )


def calculate_score(marks, assessment, attendance):
    return (
        0.5 * marks
        + 0.3 * assessment
        + 0.2 * attendance
    )


def rank_key(candidate):
    return (
        -candidate["score"],
        -candidate["marks"],
        -candidate["assessment"],
        -candidate["attendance"],
        candidate["name"].lower()
    )


def main():
    candidates = [
        {
            "name": "Alice",
            "marks": 85,
            "assessment": 90,
            "attendance": 95
        },
        {
            "name": "Bob",
            "marks": 75,
            "assessment": 80,
            "attendance": 78
        },
        {
            "name": "Charlie",
            "marks": 60,
            "assessment": 50,
            "attendance": 80
        },
        {
            "name": "David",
            "marks": 58
```

OUTPUT

```
RANKED QUALIFIED CANDIDATES
1 Eva - Score: 91.5
2 Alice - Score: 88.5
3 Bob - Score: 77.1
4 Charlie - Score: 61.0
\nINELIGIBLE CANDIDATES
David
>
```

Conclusion

- The matrix makes student-wise and subject-wise analysis simple and helps identify strong performers and weak subjects.

CASE-43 — Multi-Branch Sales Matrix Comparison

Problem Statement

Store sales for several branches, days and products in nested lists.

Calculate product

totals, compare branch totals and identify the best

product in each branch.

Objectives

- Store branch-day-product sales in nested lists.
- Calculate product totals.
- Calculate total sales for each branch.
- Find the best-selling product in every branch.

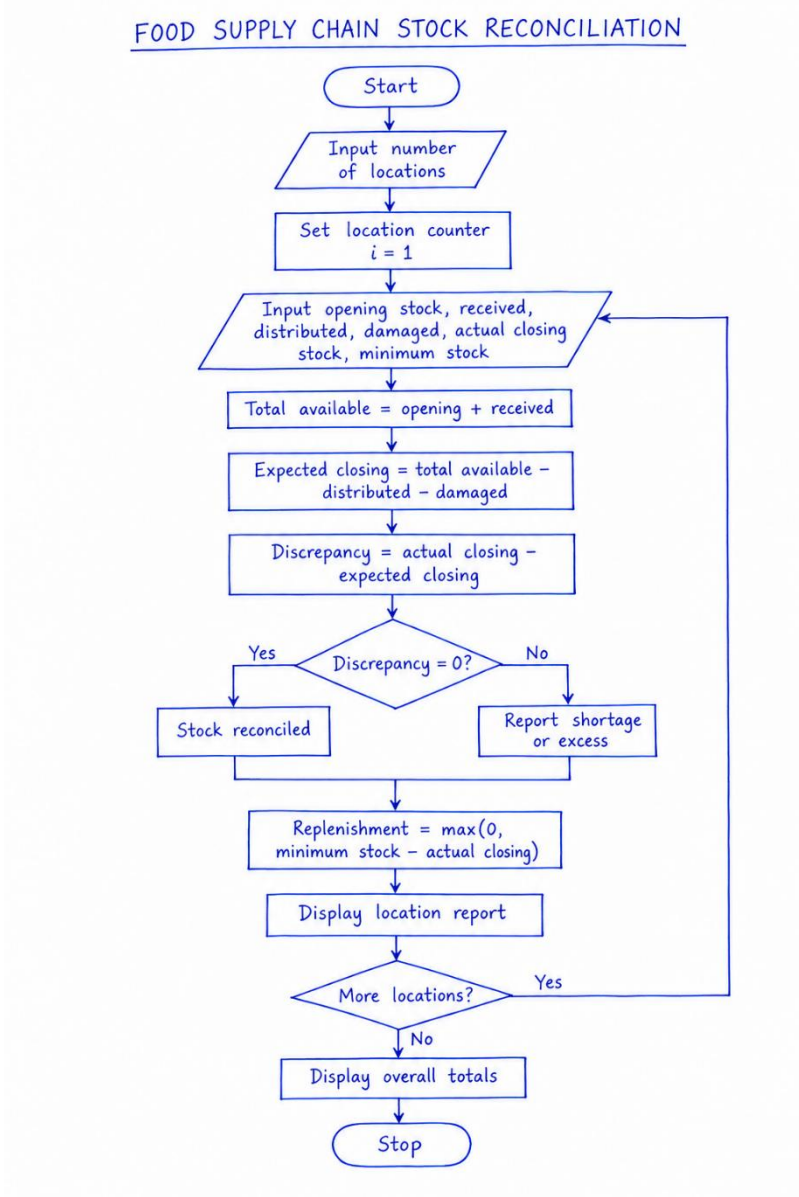
Assumptions

- Each value is a non-negative sales amount.
- Data order is branch → day → product.
- All branches use the same product list..

Algorithm

1. Create the 3-level sales list.
2. Add sales across days and products.
3. Calculate branch totals.
4. Calculate product totals for each branch.
5. Find the maximum product total in each branch.

Flow Chart


FOOD SUPPLY CHAIN STOCK RECONCILIATION
Start
Input number of locations
Set location counter i = 1
Input opening stock, received, distributed, damaged, actual closing stock, minimum stock
Total available = opening + received
Expected closing = total available − distributed − damaged
Discrepancy = actual closing − expected closing
Discrepancy = 0?
Yes
No
Stock reconciled
Report shortage or excess
Replenishment = max(0, minimum stock − actual closing)
Display location report
More locations?
Yes
No
Display overall totals
Stop

Python Source Code

```
sales = [
 [[10, 20, 15], [12, 18, 20]], # Branch A
 [[15, 12, 18], [20, 10, 25]] # Branch B
]
products = ["Pen", "Book", "Bag"]
branches = ["Branch A", "Branch B"]
branch_totals = [sum(day) for b in sales for day in []] # not used
branch_totals = [sum(sum(day) for day in b) for b in sales]
for bi, b in enumerate(sales):
 product_totals = [
 sum(day[p] for day in b) for p in range(len(products))
 ]
 best = products[product_totals.index(max(product_totals))]
 print(branches[bi], "total:", branch_totals[bi])
 print("Product totals:", dict(zip(products, product_totals)))
 print("Best product:", best)
product_total_all = [
 sum(day[p] for b in sales for day in b)
 for p in range(len(products))
]
print("Overall product totals:", dict(zip(products, product_total_all)))
```

Test Cases

1. Branch total is sum of all its days/products
.
2. Best product is the maximum product total.
3. Overall product totals combine all

Screenshot of code and output

Code

```
locations = int(input("Enter number of locations: "))

total_received = 0
total_distributed = 0
total_damaged = 0
total_replenishment = 0

for i in range(1, locations + 1):
    print("\nLocation", i)

    name = input("Enter location name: ")
    opening = float(input("Enter opening stock (kg): "))
    received = float(input("Enter received stock (kg): "))
    distributed = float(input("Enter distributed stock (kg): "))
    damaged = float(input("Enter damaged stock (kg): "))
    actual = float(input("Enter actual closing stock (kg): "))
    minimum = float(input("Enter minimum stock level (kg): "))

    if min(opening, received, distributed,
           damaged, actual, minimum) < 0:
        print("Invalid input: quantities cannot be negative.")
        print("Please run the program again.")
        break

    available = opening + received
    expected = available - distributed - damaged

    discrepancy = actual - expected
    replenishment = max(0, minimum - actual)

    print("\n--- Stock Reconciliation Report ---")
    print("Location:", name)
    print("Total available stock:", available, "kg")
    print("Expected closing stock:", expected, "kg")
    print("Actual closing stock:", actual, "kg")
    print("Discrepancy:", discrepancy, "kg")

    if discrepancy == 0:
        print("Status: Stock reconciled")
    elif discrepancy < 0:
        print("Status: Stock shortage")
    else:
        print("Status: Excess stock")

    print("Replenishment required:", replenishment, "kg")

    total_received += received
    total_distributed += distributed
```

Output

```
Enter number of locations: 5

Location 1
Enter location name: Vijayawada
Enter opening stock (kg): 5
Enter received stock (kg): 3
Enter distributed stock (kg): 6
Enter damaged stock (kg): 1
Enter actual closing stock (kg): 7
Enter minimum stock level (kg): 8

--- Stock Reconciliation Report ---
Location: Vijayawada
Total available stock: 8.0 kg
Expected closing stock: 1.0 kg
Actual closing stock: 7.0 kg
Discrepancy: 6.0 kg
Status: Excess stock
Replenishment required: 1.0 kg

Location 2
Enter location name: poranki
Enter opening stock (kg): 2
Enter received stock (kg): 5
Enter distributed stock (kg): 6
Enter damaged stock (kg): 7
Enter actual closing stock (kg): 8
Enter minimum stock level (kg): 9

--- Stock Reconciliation Report ---
Location: poranki
Total available stock: 7.0 kg
Expected closing stock: -6.0 kg
Actual closing stock: 8.0 kg
Discrepancy: 14.0 kg
Status: Excess stock
Replenishment required: 1.0 kg

Location 3
Enter location name: one towm
Enter opening stock (kg): 4
Enter received stock (kg): 7
Enter distributed stock (kg): 2
Enter damaged stock (kg): 19
Enter actual closing stock (kg): 45
Enter minimum stock level (kg): 4

--- Stock Reconciliation Report ---
Location: one towm
```

Conclusion

- Nested lists are useful for multi-dimensional sales data and make branch and product comparisons easy.

CASE-44 — Hospital Ward Occupancy Monitoring

Problem Statement

Analyse bed occupancy counts for wards over several days. Calculate utilization percentages, detect capacity violations and find the peak-demand period.

Objectives

- Store ward-wise daily occupancy.
- Calculate utilization percentage.
- Detect occupancy above ward capacity.
- Find the highest-demand ward/day.

Assumptions

- Each row represents one ward.
- Each column represents one day.
- Capacity is fixed for each ward.
- Utilization = occupied / capacity × 100.

Algorithm

1. Store occupancy matrix and ward capacities.
2. Calculate utilization for every ward/day.
3. If occupancy > capacity, report a violation.
4. Find the maximum occupancy value and its location.
5. Display utilization and peak demand.

Python Source Code

```
wards = ["Ward A", "Ward B"]
capacity = [20, 15]
occupancy = [
 [18, 21, 19],
 [10, 14, 15]
]
days = ["Mon", "Tue", "Wed"]
peak = (0, "", "")
for i in range(len(wards)):
 for j in range(len(days)):
 used = occupancy[i][j]
 util = used / capacity[i] * 100
 status = "VIOLATION" if used > capacity[i] else "OK"
 print(wards[i], days[j], round(util, 1), "%", status)
 if used > int(peak[0] or 0):
 peak = (used, wards[i], days[j])
print("Peak demand:", peak)
```

Test Cases

1. 18/20 → 90% and OK.
2. 21/20 → 105% and capacity violation.
3. Maximum occupancy 21 → Peak period Ward A, Tue.

Screenshot of Code and Output

Code

```
data = []
locations = int(input("Enter number of locations: "))

lower = float(input("Enter lower threshold: "))
upper = float(input("Enter upper threshold: "))

if locations <= 0 or lower > upper:
    print("Invalid number of locations or thresholds.")

else:
    for i in range(locations):
        name = input("\nEnter location name: ")
        values = input(
            "Enter readings separated by commas: "
        )
        readings = [float(x.strip())
                    for x in values.split(",")]

        if not readings:
            print("No readings entered.")
            continue

        data.append([name, readings])

    total = []
    counts = {"Low": 0, "Normal": 0, "High": 0}

    for name, readings in data:
        avg = sum(readings) / len(readings)
        maximum = max(readings)
        minimum = min(readings)

        print("\nLocation:", name)
        print("Readings:", readings)
        print("Average:", round(avg, 2))
        print("Maximum:", maximum)
        print("Minimum:", minimum)

        for value in readings:
            if value < lower:
                category = "Low"
            elif value <= upper:
                category = "Normal"
            else:
                category = "High"

            counts[category] += 1
            print(value, "->", category)
```

Ouput

```
Enter number of locations: 2

Location 1
Enter location name: banglore
Enter opening stock (kg): 2
Enter received stock (kg): 2
Enter distributed stock (kg): 3
Enter damaged stock (kg): 5
Enter actual closing stock (kg): 6
Enter minimum stock level (kg): 7

--- Stock Reconciliation Report ---
Location: banglore
Total available stock: 4.0 kg
Expected closing stock: -4.0 kg
Actual closing stock: 6.0 kg
Discrepancy: 10.0 kg
Status: Excess stock
Replenishment required: 1.0 kg

Location 2
Enter location name: pune
Enter opening stock (kg): 4
Enter received stock (kg): 56
Enter distributed stock (kg): 78
Enter damaged stock (kg): 23
Enter actual closing stock (kg): 12
Enter minimum stock level (kg): 43

--- Stock Reconciliation Report ---
Location: pune
Total available stock: 60.0 kg
Expected closing stock: -41.0 kg
Actual closing stock: 12.0 kg
Discrepancy: 53.0 kg
Status: Excess stock
Replenishment required: 31.0 kg

--- Overall Summary ---
Total received: 58.0 kg
Total distributed: 81.0 kg
Total damaged: 28.0 kg
Total replenishment required: 32.0 kg
```

Conclusion

- The program quickly identifies overcrowding and peak demand, helping hospital staff monitor bed usage.

CASE-45 — Weekly Electricity Consumption Matrix

- **Problem Statement**
- **Process appliance-wise electricity readings for seven days. Calculate daily totals,**
- **appliance totals and identify values exceeding a**
- **specified daily limit.**

Ojectives

- Store appliance readings for 7 days.
- Calculate each day's total consumption.
- Calculate each appliance's weekly total.
- Flag consumption above a limit.

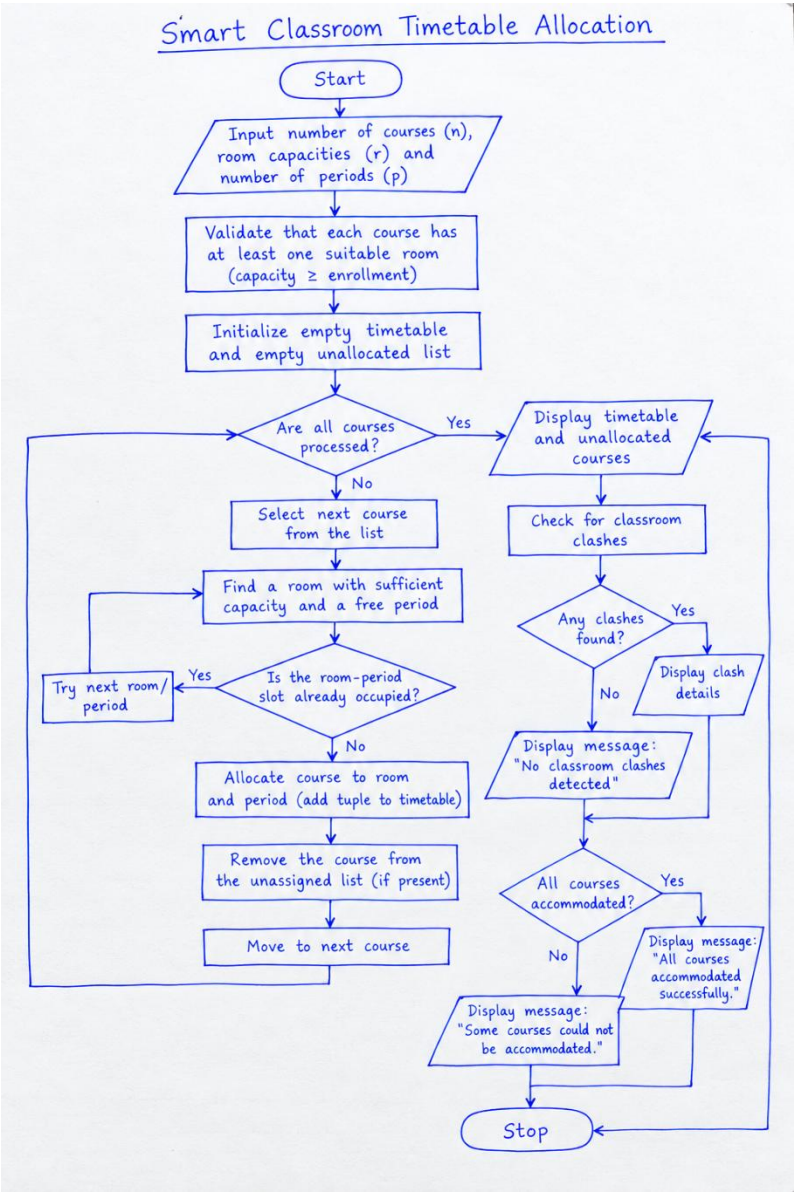
Assumptions

- Each row represents one appliance.
- Each column represents one day.
- Readings are in kWh.
- Any daily total above the limit is flagged.

Algorithm

1. Create the appliance × 7-day matrix.
2. Sum each column for daily totals.
3. Sum each row for appliance totals.
4. Compare daily totals with the limit.
5. Display totals and warnings.

Flowchart


Smart Classroom Timetable Allocation
Start
Input number of courses (n), room capacities (r) and number of periods (p)
Validate that each course has at least one suitable room (capacity ≥ enrollment)
Initialize empty timetable and empty unallocated list
Are all courses processed?
Yes
No
Select next course from the list
Find a room with sufficient capacity and a free period
Is the room-period slot already occupied?
Yes
Try next room/ period
No
Allocate course to room and period (add tuple to timetable)
Remove the course from the unassigned list (if present)
Move to next course
Display timetable and unallocated courses
Check for classroom clashes
Any clashes found?
Yes
Display clash details
No
Display message: "No classroom clashes detected"
All courses accommodated?
Yes
Display message: "All courses accommodated successfully."
No
Display message: "Some courses could not be accommodated."
Stop

Python Source Code

```
appliances = ["Fan", "AC", "Light"]
days = ["Mon","Tue","Wed","Thu","Fri","Sat","Sun"]
use = [
 [2,2,3,2,3,2,2],
 [8,9,10,8,9,11,10],
 [1,1,1,2,1,1,1]
]
limit = 15
daily = [sum(use[i][d] for i in range(len(appliances)))
 for d in range(7)]
appliance_total = [sum(row) for row in use]
print("Daily totals:", daily)
print("Appliance totals:",
 dict(zip(appliances, appliance_total)))
for d, total in zip(days, daily):
 if total > limit:
 print(d, "EXCEEDS LIMIT:", total, "kWh")
```

Test Cases

1. Daily total ≤ 15 → Normal.
2. Daily total > 15 → Warning.
3. AC total is the largest appliance total.

Screenshot of code and output

Code

```
lef is_eligible(marks, assessment, attendance):
    return (
        marks >= 60
        and assessment >= 50
        and attendance >= 75
    )


lef calculate_score(marks, assessment, attendance):
    return (
        0.5 * marks
        + 0.3 * assessment
        + 0.2 * attendance
    )


lef rank_key(candidate):
    return (
        -candidate["score"],
        -candidate["marks"],
        -candidate["assessment"],
        -candidate["attendance"],
        candidate["name"].lower()
    )


lef main():
    candidates = [
        {
            "name": "Alice",
            "marks": 85,
            "assessment": 90,
            "attendance": 95
        },
        {
            "name": "Bob",
            "marks": 75,
            "assessment": 80,
            "attendance": 78
        },
        {
            "name": "Charlie",
            "marks": 60,
            "assessment": 50,
            "attendance": 80
        },
        {
            "name": "David",
            "marks": 58
```

Output

```
///
    ==============================
    RANKED QUALIFIED CANDIDATES
    1 Eva - Score: 91.5
    2 Alice - Score: 88.5
    3 Bob - Score: 77.1
    4 Charlie - Score: 61.0
    \nINELIGIBLE CANDIDATES
    David
>>>
```

Conclusion

- The matrix helps track weekly electricity use and quickly identify high-consumption days or appliances

Problem Statement

Q.46 — Inventory Matrix Reconciliation Problem Statement: Compare product quantities across warehouses, identify shortages and surpluses, and calculate transfers needed to reach target stock levels..

Objectives

- Store warehouse-wise product quantities.
- Compare stock with target quantities.
- Identify shortages and surpluses.
- Calculate simple transfer requirements.

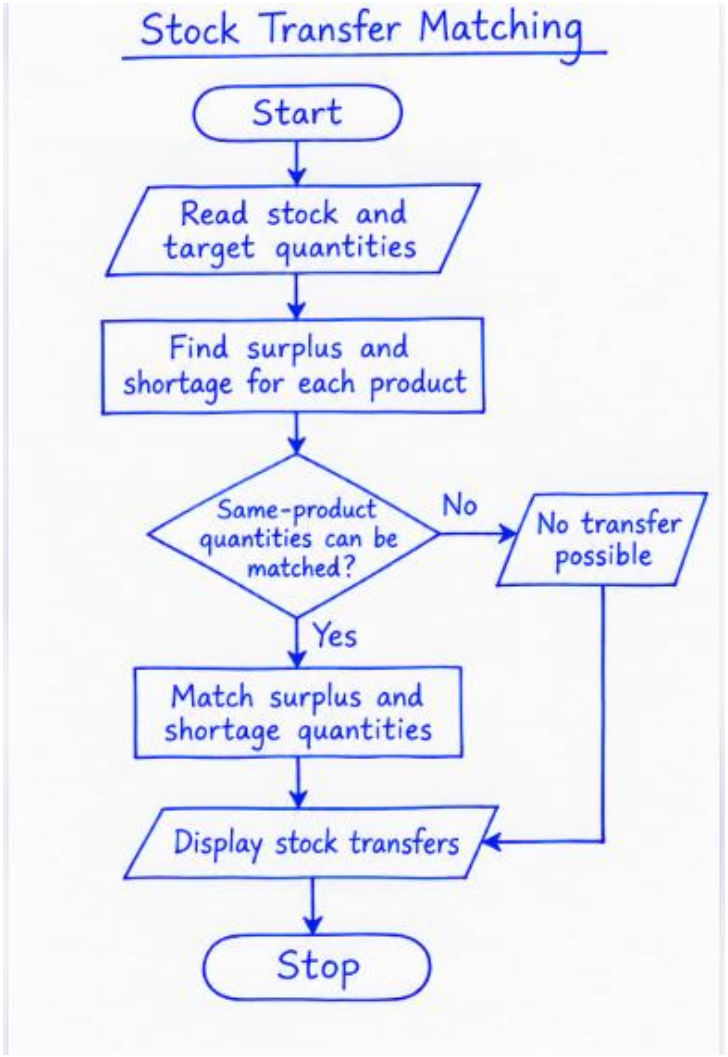
Assumptions

- Rows are warehouses and columns are products
- Target is the required stock per warehouse.
- Positive difference means surplus; negative means shortage.
- Transfers are shown as quantity to move, not shipment ro

Algorithm

- Create stock matrix and target matrix.
- Calculate stock − target for each product.
- Report positive values as surplus and negative values as shortage.
- Match surplus with shortage for the same product.
- Display transfer quantities.

Flow chart


Stock Transfer Matching
Start
Read stock and target quantities
Find surplus and shortage for each product
Same-product quantities can be matched?
No
No transfer possible
Yes
Match surplus and shortage quantities
Display stock transfers
Stop

Python source code

```
warehouses = ["W1", "W2"]
 products = ["Pen", "Book"]
stock = [[50, 20], [10, 40]]
 target = [[30, 25], [25, 30]]
surplus = [[], []]
shortage = [[], []]
 for i in range(2):
   for j in range(2):
  diff = stock[i][j] - target[i][j]
  if diff > 0:
surplus[i].append((products[j], diff))
elif diff < 0:
shortage[i].append((products[j], -diff))
print("Surplus:", dict(zip(warehouses, surplus)))
print("Shortage:", dict(zip(warehouses, shortage)))
 for p in range(2):
s = sum(max(stock[i][p] - target[i][p], 0) for i in range(2))
  sh = sum(max(target[i][p] - stock[i][p], 0) for i in range(2))
print(products[p], "transfer needed:", min(s, sh))
```

Test Cases

W1 Pen 50 vs 30 → Surplus 20.
2. W1 Book 20 vs 25 → Shortage 5.
3. Same product surplus can cover same-product shortage.

Screenshot of code and output

Code

```
accounts = {
    "A101": 1000.0,
    "A102": 500.0
}

transactions = [
    ("DEPOSIT", "A101", 200),
    ("WITHDRAW", "A101", 100),
    ("TRANSFER", "A101", "A102", 150),
    ("WITHDRAW", "A101", 100),
    ("WITHDRAW", "A101", 100),
    ("WITHDRAW", "A101", 5000),
    ("DEPOSIT", "A102", -50)
]

history = []
pattern_count = {}
suspicious = []

for txn in transactions:
    action = txn[0]
    valid = False
    pattern = None

    if action == "DEPOSIT":
        _, account, amount = txn
        pattern = (action, account, amount)

        if account in accounts and amount > 0:
            accounts[account] += amount
            valid = True

    elif action == "WITHDRAW":
        _, account, amount = txn
        pattern = (action, account, amount)

        if account not in accounts or amount <= 0:
            pass
        elif accounts[account] >= amount:
            accounts[account] -= amount
            valid = True

    elif action == "TRANSFER":
        _, source, target, amount = txn
        pattern = (action, source, target, amount)

        if (source in accounts and target in accounts
                and source != target and amount > 0
```

Output

- Surplus: {'W1': [('Pen', 20)],
- 'W2': [('Book', 10)]}
- Shortage: {'W1': [('Book', 5)], 'W2': [('Pen', 15)]}
- Pen transfer needed: 15 | Book t

Conclusion

- The program reconciles warehouse stock with targets and gives a simple quantity for balancing inventory..

Q.47 — Sports Tournament Ranking System

Problem Statement

Store match results and team scores, calculate points and a tie-break value, then rank teams according to a simple competition policy.

Objectives

- Store match results.
- Give 3 points for a win and 1 for a draw.
- Calculate tie-break values.
- Produce the final ranking.

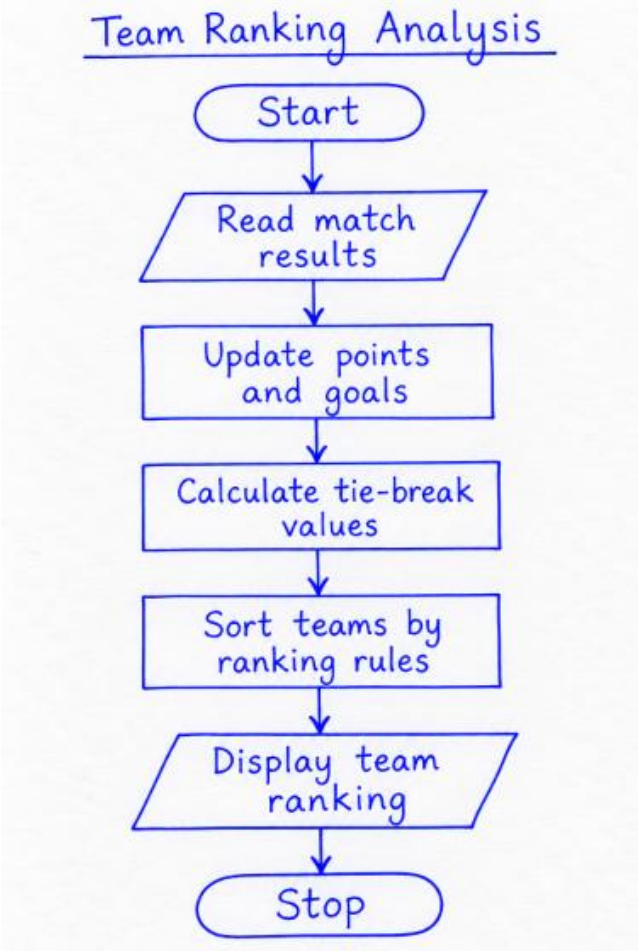
Assumptions

- Win = 3 points, draw = 1 point, loss = 0
- Tie-break is goal difference (goals for − goals against).
- Teams are ranked by points, then goal difference, then goals scored.

Algorithm

- Create match tuples: (team1, goals1, team2, goals2).
- Update points and goals for every match.
- Calculate goal difference.
- Sort by points, goal difference and goals scored.
- Display ranking.

Flowchart


Team Ranking Analysis
Start
Read match results
Update points and goals
Calculate tie-break values
Sort teams by ranking rules
Display team ranking
Stop

Python code

```
matches = [    ("A", 2, "B", 1),
 ("C", 1, "A", 1),
  ("B", 2, "C", 0) ]
teams = ["A", "B", "C"]
data = {t: [0,0,0]
for t in teams}  # points, GF, GA
 for t1, g1, t2, g2 in matches:
data[t1][1] += g1;
data[t1][2] += g2
 data[t2][1] += g2; data[t2][2] += g1
    if g1 > g2:
 data[t1][0] += 3
 elif g2 > g1:
 data[t2][0] += 3
  else:
   data[t1][0] += 1; data[t2][0] += 1
ranking = sorted(
teams,
  key=lambda t: (data[t][0], data[t][1]-data[t][2], data[t][1]),
 reverse=True )
for rank, t in enumerate(ranking, 1):
   gd = data[t][1] - data[t][2]
print(rank, t, "Points:", data[t][0], "GD:", gd
```

Test cases

. A beats B → A gets 3 points.
A-C draw → both get 1 point.
Equal points → higher goal difference ranks first.

Screenshot of Code and Output

CODE

```
key = [
    x.strip().upper()
    for x in input(
        "Enter answer key separated by commas: "
    ).split(",")
]

if not key or any(x == "" for x in key):
    print("Invalid answer key.")
else:
    students = int(input("Enter number of students: "))

    if students <= 0:
        print("Number of students must be positive.")
    else:
        correct = [0] * len(key)
        total = [0] * len(key)

        for i in range(students):
            while True:
                answers = [
                    x.strip().upper()
                    for x in input(
                        f"Student {i + 1} answers: "
                    ).split(",")
                ]

                if len(answers) == len(key):
                    break

                print("Enter exactly", len(key), "answers.")

            for j in range(len(key)):
                total[j] += 1
                if answers[j] == key[j]:
                    correct[j] += 1

        rates = []

        print("\n--- QUESTION-WISE ANALYSIS ---")

        for j in range(len(key)):
            rate = correct[j] / total[j] * 100
            missed = total[j] - correct[j]
            rates.append(rate)

            if rate >= 80:
                category = "Excellent"
```

OUTPUT

- A Points: 4 GD: 1
- 2 B Points: 3 GD: 0
- 3 C Points: 1 GD: -1

Conclusion

- The program demonstrates tuples, dictionaries, sorting and tie-break rules for a simple tournament ranking system.

48 — Tuple-Based Employee Shift Conflict Detection

Problem Statement

Represent employee shifts as tuples, compare time intervals, detect overlapping shifts and identify employees with conflicting schedules.

Objectives

- Store shifts as tuples.
- • Compare start and end times.
- • Detect overlapping shifts for the same employee.
- • List employees with conflicts.

Assumptions

- Shift tuple format is (employee, start, end).
- • Times are whole hours from 0 to 24.
- • Two shifts conflict when start1 < end2 and start2 < end1.

Algorithm

- Start the program.
- Store all shifts as tuples.
- 2. Compare every pair of shifts.
- 3. Check whether the employee names match.
- 4. Use interval-overlap condition.
- 5. Add conflicting employees to a set and display them.

Python code

```
# Multi-Branch Retail Inventory Transfer

branches = {
    "Branch A": {"Rice": 80, "Oil": 20},
    "Branch B": {"Rice": 25, "Oil": 60},
    "Branch C": {"Rice": 45, "Oil": 30}
}

targets = {
    "Rice": 50,
    "Oil": 40
}

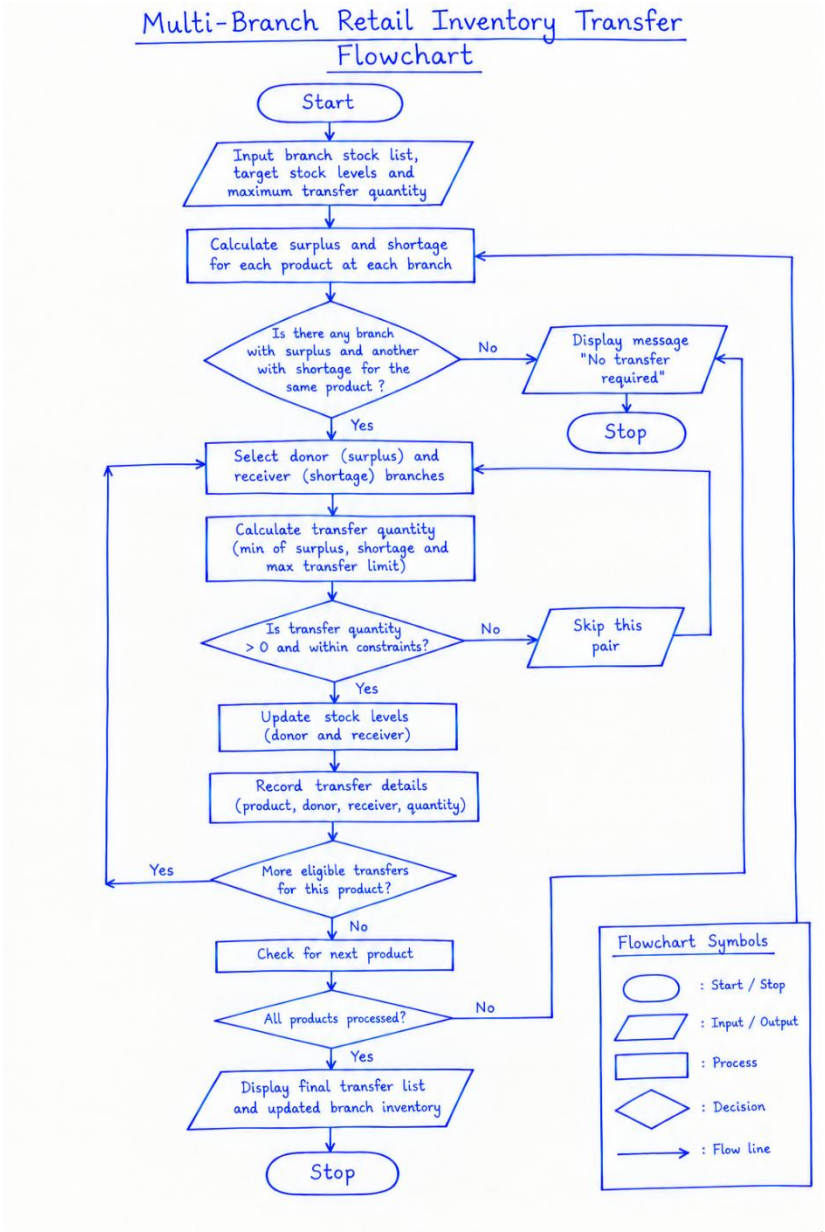
max_transfer = 15
transfers = []

# Process each product
for product, target in targets.items():

    surplus = {}
    shortage = {}

    # Calculate surplus and shortage
    for branch, stock in branches.items():
```

Flowchart


Multi-Branch Retail Inventory Transfer Flowchart
Start
Input branch stock list, target stock levels and maximum transfer quantity
Calculate surplus and shortage for each product at each branch
Is there any branch with surplus and another with shortage for the same product ?
No
Display message "No transfer required"
Stop
Yes
Select donor (surplus) and receiver (shortage) branches
Calculate transfer quantity (min of surplus, shortage and max transfer limit)
Is transfer quantity > 0 and within constraints?
No
Skip this pair
Yes
Update stock levels (donor and receiver)
Record transfer details (product, donor, receiver, quantity)
More eligible transfers for this product?
Yes
No
Check for next product
All products processed?
No
Yes
Display final transfer list and updated branch inventory
Stop
Flowchart Symbols
: Start / Stop
: Input / Output
: Process
: Decision
: Flow line

Test Cases

1 Asha 9–13 and 12–16 → Conflict
. 2. Bala 9–12 and 13–17 → No conflict.
3. Different employees with overlap → Ignore.

Screenshot of code and output

CODE

```
branches = {
    "Branch A": {"Rice": 80, "Oil": 20},
    "Branch B": {"Rice": 25, "Oil": 60},
    "Branch C": {"Rice": 45, "Oil": 30}
}

targets = {
    "Rice": 50,
    "Oil": 40
}

max_transfer = 15
transfers = []

# Process each product
for product, target in targets.items():

    surplus = {}
    shortage = {}

    # Calculate surplus and shortage
    for branch, stock in branches.items():
        quantity = stock.get(product, 0)

        if quantity > target:
            surplus[branch] = quantity - target
        elif quantity < target:
            shortage[branch] = target - quantity

    # Transfer stock between branches
    for donor in list(surplus.keys()):
        for receiver in list(shortage.keys()):

            if surplus[donor] <= 0:
                break

            if shortage[receiver] <= 0:
                continue

            quantity = min(
                surplus[donor],
                shortage[receiver],
                max_transfer
            )

            if quantity > 0:
                # Update inventory
                branches[donor][product] -= quantity
```

OUTPUT

- Conflict: ('Asha', 9, 13) and ('Asha', 12, 16)
- Employees with conflicts: {'Asha'}

Conclusion

- Tuple records and interval comparison provide a simple way to detect employee scheduling conflicts..

Q.49 — Tuple-Based Product Catalogue Audit

Problem Statement

Store product ID, price and stock quantity in tuples. Detect invalid records, calculate inventory value and identify products needing replenishment.

Objectives

•Store product records as tuples.
•Detect invalid IDs, prices or stock values.
•Calculate inventory value = price × stock. • I
•dentify items below the reorder level.

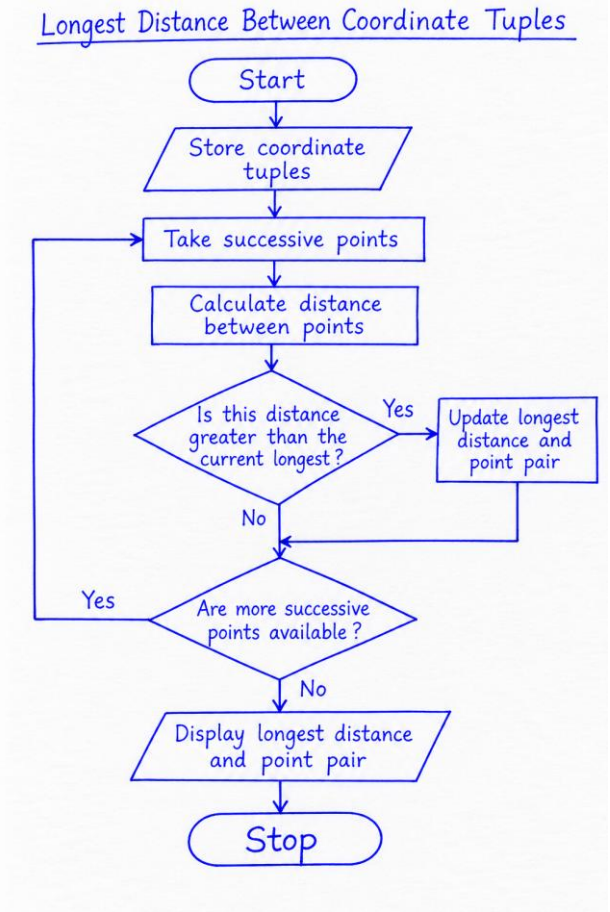
Assumptions

- Tuple format is (product_id, price, quantity).
- Price and quantity must be non-negative.
- Product ID must not be empty.
- Replenishment is needed when quantity < 5.

Algorithm

•Read product tuples.
• Validate ID, price and quantity.
•Skip invalid records and report them.
•. Calculate price × quantity for valid records.
•List products with stock below 5.

Flow chart


Longest Distance Between Coordinate Tuples
Start
Store coordinate tuples
Take successive points
Calculate distance between points
Is this distance greater than the current longest?
Yes
Update longest distance and point pair
No
Are more successive points available?
Yes
No
Display longest distance and point pair
Stop

Python source code

```
products = [
("P101", 50, 10),
 ("P102", -20, 5),
("P103", 80, 2),
("P104", 30, 0)
]
value = 0
reorder = []
invalid = []
for pid, price, qty in products:
 if not pid or price < 0 or qty < 0:
invalid.append((pid, price, qty))
Continue
 value += price * qty
if qty < 5:
 reorder.append(pid)
print("Invalid records:", invalid)
print("Inventory value:", value)
print("Replenishment needed:", reorder)
```

Test Cases

Negative price → Invalid record. 2. P103 quantity 2 → Replenishment needed. 3. P101 value = 50 × 10 = 500.

Screenshots of code and output

CODE

```
# Student groups: (group_id, student_ids, category)
groups = [
    ("G1", ["S101", "S102"], "UG"),
    ("G2", ["S103", "S104", "S105"], "UG"),
    ("G3", ["S106", "S107"], "PG"),
    ("G4", ["S108", "S109", "S110"], "PG"),
    ("G5", ["S102", "S111"], "UG")
]

# Room details: capacity and eligible categories
rooms = {
    "Room A": {"capacity": 3, "eligible": ["UG"]},
    "Room B": {"capacity": 2, "eligible": ["PG"]},
    "Room C": {"capacity": 4, "eligible": ["UG", "PG"]}
}

# Initialize allocation records
occupancy = {room: [] for room in rooms}
allocations = []
seen_students = set()
assigned_students = set()
duplicate_ids = set()

# Allocate each group
for group_id, students, category in groups:

    # Detect duplicate student IDs across groups
    duplicates = [
        student for student in students
        if student in seen_students
    ]

    if duplicates:
        duplicate_ids.update(duplicates)
        print("Duplicate assignment detected:",
              group_id, duplicates)
        continue

    seen_students.update(students)
    allocated = False

    # Find a room satisfying eligibility and capacity
    for room, details in rooms.items():
        available = (
            details["capacity"] - len(occupancy[room])
        )

        if (category in details["eligible"]
```

OUTPUT

- Invalid records: [('P102', -20, 5)]
  Inventory value: 740
  Replenishment needed: ['P103', 'P104']

Conclusion

The tuple-based audit checks catalogue quality and quickly identifies inventory value and low-stock products.

Q.50 — Tuple-Based Route Distance Analysis

Problem Statement

Store geographical points as coordinate tuples, calculate distances between successive points using a supplied formula, and identify the longest route segment.

Objectives

•Store route points as tuples.
•Calculate distance between consecutive points.
• Display every segment distance.
• Find the longest segment.

Assumptions

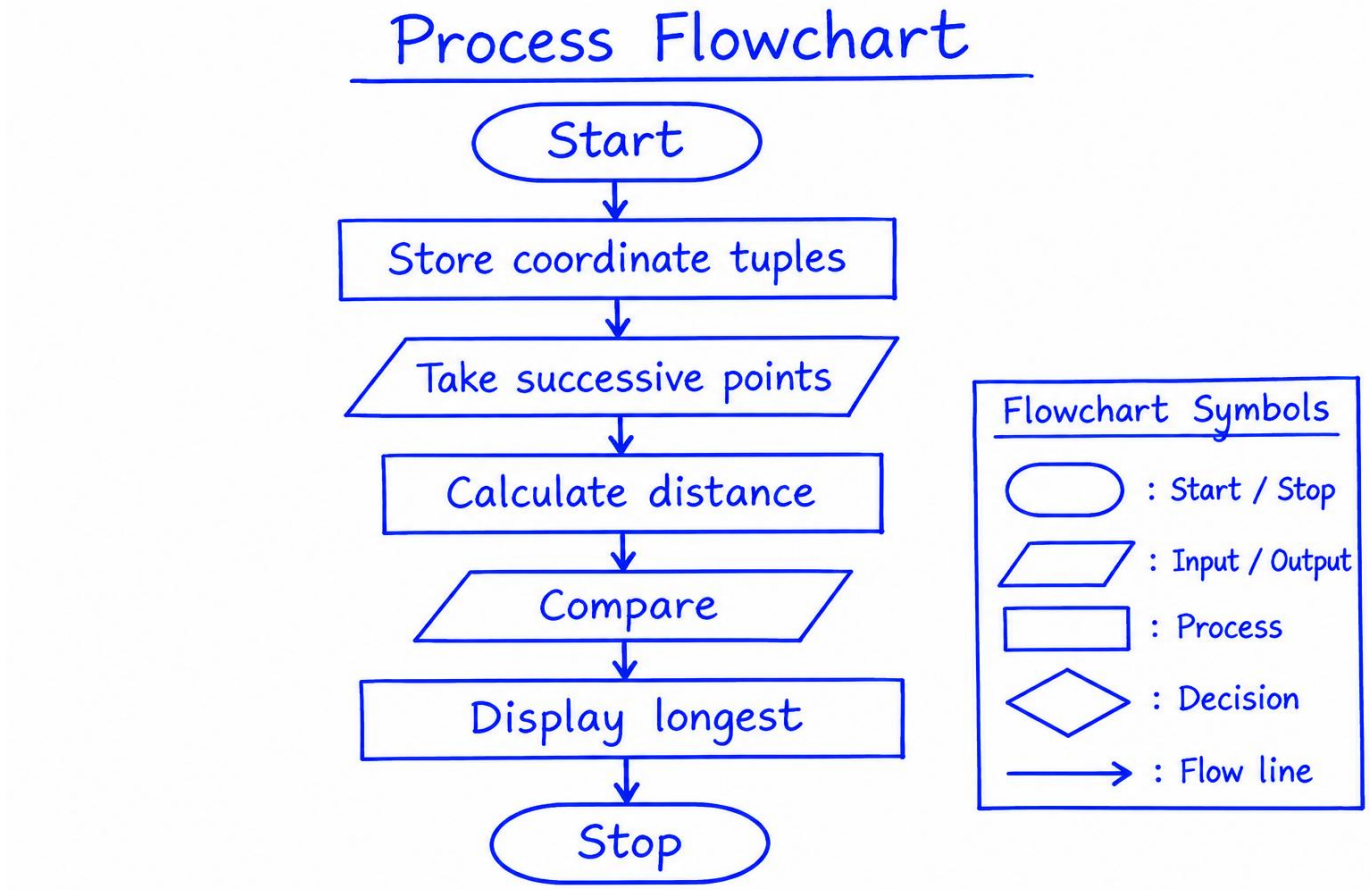
- Each point is stored as (x, y).
- • Euclidean distance is used: $\sqrt{((x2-x1)^2 + (y2-y1)^2)}$.
- • At least two points are supplied.

Algorithm

- 1. Create the coordinate tuple list.
- 2. Take each pair of successive points.
- 3. Apply the distance formula.
- 4. Compare all segment distances.

- 5. Display the longest segment.

Flow chart


Process Flowchart
Start
Store coordinate tuples
Take successive points
Calculate distance
Compare
Display longest
Stop
Flowchart Symbols
: Start / Stop
: Input / Output
: Process
: Decision
: Flow line

Python source code

- import math
- points = [(0, 0), (3, 4), (6, 4), (6, 8)]

```
 distances = [] for i in range(len(points) - 1):
 x1, y1 = points[i]
x2, y2 = points[i + 1]
d = math.sqrt((x2-x1)**2 + (y2-y1)**2)
 distances.append(d
    print(points[i], "to", points[i+1], "=", round(d, 2))
m = max(distances)
i = distances.index(m)
print("Longest segment:", points[i], "to", points[i+1])
print("Distance:", round(m, 2))
```

Test cases

(0,0)→(3,4) → Distance 5.
2. (3,4)→(6,4) → Distance 3.
3. Longest of 5, 3, 4 → First segment.

Screenshot of code and output

CODE

```
buildings = {
    "Academic Block": {
        "electricity": [120, 135, 150],
        "water": [18, 20, 24]
    },
    "Library": {
        "electricity": [80, 78, 90],
        "water": [12, 11, 13]
    },
    "Hostel": {
        "electricity": [200, 220, 250],
        "water": [35, 40, 45]
    }
}

# Rates and daily consumption thresholds
electricity_rate = 8
water_rate = 30
electricity_limit = 140
water_limit = 25

results = []

for name, data in buildings.items():
    electricity = data["electricity"]
    water = data["water"]

    total_e = sum(electricity)
    total_w = sum(water)

    avg_e = total_e / len(electricity)
    avg_w = total_w / len(water)

    cost_e = total_e * electricity_rate
    cost_w = total_w * water_rate
    total_cost = cost_e + cost_w

    print("\n---", name, "---")
    print("Total electricity:", total_e, "kWh")
    print("Total water:", total_w, "kL")
    print("Average electricity:", round(avg_e, 2))
    print("Average water:", round(avg_w, 2))
    print("Electricity cost: Rs.", cost_e)
    print("Water cost: Rs.", cost_w)
    print("Total cost: Rs.", total_cost)

    # Calculate daily percentage changes
    for label, readings in [
        ("Electricity", electricity),
        ("Water", water)
```

OUTPUT

- (0, 0) to (3, 4) = 5.0
- (3, 4) to (6, 4) = 3.0
- (6, 4) to (6, 8) = 4.0
- Longest segment: (0, 0) to (3, 4)
  Distance: 5.0

Conclusion

- Tuple coordinates and the distance formula make it easy to analyse route segments and find the longest one.